

SAHAY

- Rainfall Analysis & Flood Prediction

A PROJECT REPORT

Submitted by

Kavya	(22BCE10385)
Simarpreet Singh	(22BCE10914)
Sneha Mishra	(22BCE10932)
Pooja Prajapat	(22BCE10984)
Ashish Kumar	(22BCE11353)

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING



SCHOOL OF COMPUTING SCIENCE AND ENGINEERING

VIT BHOPAL UNIVERSITY

**KOTHRICALAN, SEHORE
MADHYA PRADESH – 466114**

April 2024

**VIT BHOPAL UNIVERSITY, KOTHRIKALAN, SEHORE
MADHYA PRADESH – 466114**

BONAFIDE CERTIFICATE

Certified that this project report titled “**Sahay - Rainfall Analysis and Flood Prediction**” is the Bonafide work of **Kavya (22BCE10385), Simarpreet Singh (22BCE10914), Sneha Mishra (22BCE10932), Pooja Prajapat (22BCE10984), Ashish Kumar (22BCE11353)** who carried out the project work under my supervision. Certified further that to the best of my knowledge the work reported here does not form part of any other project / research work on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

PROJECT SUPERVISOR

Dr. Vijendra Singh Bramhe, Senior Assistant Professor
School of Computing Science and Engineering
VIT BHOPAL UNIVERSITY

The Project Exhibition II Examination is held on 29th April, 2024.

ACKNOWLEDGEMENT

First and foremost, we would like to thank the Lord Almighty for his presence and immense blessings throughout the project work.

We wish to express our heartfelt gratitude to Dr. S. Poonkuntran, Head of the Department, School of Computing Science and Engineering for much of his valuable support and encouragement in carrying out this work.

We would like to thank our internal guide Dr. Vijendra Singh Bramhe, for continually guiding and actively participating in my project, giving valuable suggestions to complete the project work.

We would like to thank all the technical and teaching staff of the School of Computing Science and Engineering, who extended directly or indirectly all their support.

Last, but not the least, we are deeply indebted to our parents who have been the greatest support.

LIST OF ABBREVIATIONS

Abbreviation	Meaning
ML	Machine Learning
ARIMA	Autoregressive Integrated Moving Average.
NaN	Not a Number
MAE	Mean Absolute Error
LDA	Linear Discriminant Analysis
RNN	Recurrent Neural Network
CSV	Comma Separated Values
HTTP	HyperText Transfer Protocol
HTML	HyperText Markup Language
SMOTE	Synthetic Minority Over-sampling TEchnique

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
1	Figure 1: index.html	39
2	Figure 2: about.html	39
3	Figure 3: rainfall_entry.html	40
4	Figure 4: rainfall_result.html	40
5	Figure 5: flood_entry.html	41
6	Figure 6: flood_result.html	41
7	Figure 7: Mean Absolute Error	43

ABSTRACT

Floods are among the most destructive natural disasters, which are highly complex to model. The research on the advancement of flood prediction models contributed to risk reduction, policy suggestion, minimization of the loss of human life, and reduction of the property damage associated with floods. To mimic the complex mathematical expressions of physical processes of floods, during the past two decades, machine learning (ML) methods contributed highly to the advancement of prediction systems providing better performance and cost-effective solutions. Due to the vast benefits and potential of ML, its popularity dramatically increased among hydrologists. Researchers, through introducing novel ML methods and hybridizing of the existing ones aim at discovering more accurate and efficient prediction models. The main contribution of this is to demonstrate the state of the art of ML models in flood prediction. The performance comparison of ML model presents an in-depth understanding of the different techniques within the framework of a comprehensive evaluation and discussion. As a result, this report introduces the most promising prediction methods for both long-term and short-term floods. This survey can be used as a guideline for hydrologists as well as climate scientists in choosing the proper ML method according to the prediction task.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	List of Abbreviations	4
	List of Figures	5
	Abstract	6
1	INTRODUCTION 1.1 Introduction 1.2 Motivation for the Work 1.3 Introduction to the Project 1.4 Problem Statement 1.5 Objective of the Work 1.6 Organization of the Thesis 1.7 Summary	9 - 11
2	LITERATURE SURVEY 2.1 Introduction 2.2 Core Area of the Project 2.3 Method Used in the Project 2.4 Research Issues/Observations from Literature Survey 2.5 Summary	12 - 13
3	SYSTEM ANALYSIS 3.1 Introduction 3.2 Proposed System 3.3 Limitations 3.4 Summary	14 - 15

4	SYSTEM DESIGN AND IMPLEMENTATION 4.1 Introduction 4.2 Modules Design & Implementation 4.3 Templates Design 4.4 Technologies Used 4.5 Libraries Used 4.6 Methodologies Used 4.7 User Interface 4.8 Summary	16 - 42
5	PERFORMANCE ANALYSIS 5.1 Introduction 5.2 Performance Measures 5.3 Performance Analysis 5.4 Summary	43 - 44
6	FUTURE ENHANCEMENT AND CONCLUSION 6.1 Introduction 6.2 Limitation/Constraints of the System 6.3 Future Enhancements 6.4 Conclusion	45 - 46
	References	47

INTRODUCTION

1.1 Introduction

SAHAY is an innovative web application developed to address the urgent need for accurate flood predictions. In this, we have provided a user interface to the common public to check the level of water flow in rivers in future and have provided a mechanism of notification if there is any possibility of flood due to any river in nearby future(12 months). Along with that users can also see the historical trends of rivers flow and can visualize the rainfall patterns also in their Sub-Division area, so with that much information beforehand and knowing the chances of the flood in any region we can prepare ourselves and alert the local public so that loss would be minimum. This project report provides a detailed overview of SAHAY and its working including its motivation, problem statement, objectives, and organizational structure.

1.2 Motivation for the Work

The motivation behind the development of SAHAY stems from the devastating impact of floods on communities, economies, and the environment. Floods are among the most destructive natural disasters, causing loss of life, damage to infrastructure, and disruption of essential services. By providing accurate flood predictions, SAHAY aims to help communities better prepare for and mitigate the effects of these events.

1.3 Introduction to the Project

This project focuses on creating a user-friendly web application that predicts floods accurately using real-time data on rivers, water levels, and rainfall. The application aims to provide timely flood forecasts to help communities prepare for and respond to flood events. It integrates data from governmental sites, using advanced data analytics and machine learning to improve prediction models continuously. The application visualizes flood risk, water levels, and rainfall patterns making it easier for users to understand the information. It also includes alert systems

for notifying users in flood prone. The project's goal is to develop a web application that is effective in reducing the impact of floods on communities.

1.4 Problem Statement

Current flood prediction systems often rely on outdated or limited data sources, leading to inaccurate forecasts and inadequate preparedness measures. Also, many existing systems lack user-friendly interfaces, making it difficult for the public to access and understand the information provided. SAHAY seeks to address these issues by leveraging state-of-the-art technologies to improve the accuracy and accessibility of flood predictions.

1.5 Objective of the Work

The primary objective of SAHAY is to develop a user-friendly web application that harnesses the power of real-time data to accurately predict floods. By integrating data from various sources and utilizing advanced predictive models, the application aims to provide users with timely and reliable information to help them make informed decisions during flood events.

1.6 Organization of the Thesis

This thesis is organized into several chapters, each focusing on a specific aspect of SAHAY. Chapter 1 introduces the project and outlines its objectives. Chapter 2 reviews the existing literature on flood prediction and discusses the methodologies and technologies used in SAHAY. Chapter 3 describes the design of the system and Chapter 4 deals with implementation of the web application, including its features and functionality. Chapter 5 presents the results of the application's performance evaluation and validation. Finally, Chapter 6 concludes the thesis with a summary of the key findings and suggestions for future work.

1.7 Summary

SAHAY is a groundbreaking project that has the potential to not only revolutionize flood prediction and preparedness but also serve as tool of significant assistance during other natural calamities. By leveraging real-time data and advanced technologies, the application aims to provide accurate and timely information to help communities mitigate the impact of floods. This project report provides a comprehensive overview of SAHAY, detailing its motivation, objectives, and organizational structure.

LITERATURE REVIEW

2.1 Introduction

Flood prediction websites are essential tools for providing timely information and warnings to communities at risk of flooding. To ensure these websites are effective and user-friendly, they should be designed with principles such as user-centric design, accessibility, and responsiveness in mind. Incorporating features like real-time data integration, interactive maps, and alert systems is crucial for keeping users informed. User interface considerations, including visual design, information architecture, and feedback mechanisms, are vital for enhancing usability and user experience. Looking ahead, integrating with social media platforms and utilizing AI and machine learning algorithms can further enhance the accuracy and effectiveness of flood prediction websites. By integrating these elements into their development process, web developers can create websites that play a crucial role in saving lives and minimizing the impact of flooding events on communities.

2.2 Core Area of the Project

The focus of this project is to develop and deploy rainfall analysis and flood prediction websites that are both strong and user-friendly. This involves creating features like real-time data integration, and alert systems to provide timely information to communities at risk of flooding. It also includes making sure the website is easy to access. Design aspects like visual appeal, organization of information, and ways for users to give feedback are also important. Ultimately, the goal is to create a website that is a valuable tool for saving lives and reducing the impact of flooding on communities.

2.3 Method Used in the Project

In addition to the methods, we've already talked about, we're using a few other important techniques in our rainfall analysis and flood prediction web application.

- To predict the value of flood, we have used **Linear Discriminant Analysis (LDA)** or discriminant function analysis is a generalization of Fisher's linear discriminant, a method

used in statistics, pattern recognition and machine learning to find a linear combination of features that characterizes or separates two or more classes of objects or events.

- For analysis of rainfall, we have used **Recurrent Neural Network (RNN)**, it's a class of artificial neural network where connections between nodes form a directed graph along a temporal sequence. This allows it to exhibit temporal dynamic behavior. Unlike feedforward neural networks, RNNs can use their internal state (memory) to process sequences of inputs. This makes them applicable to tasks such as unsegmented, connected handwriting recognition or speech recognition.

2.4 Research Issues

The literature survey has identified key challenges and observations in developing a flood prediction website that integrates data on water level, rainfall, and river flow. One major challenge is integrating and processing diverse data sources, including inconsistencies in formats, quality, and availability, which impact prediction model accuracy. Another issue is the complexity and performance of prediction models, where accuracy depends heavily on model complexity and data quality. Advanced machine learning techniques like deep learning can be explored to enhance accuracy. Scalability and resource constraints require the adoption of cloud computing and scalable architectures. User interface design and accessibility are crucial for the website's effectiveness, advocating for user-centered design and interactive data visualization techniques. Data privacy and security concerns, including safeguarding sensitive data such as real-time water level and rainfall data, necessitate robust encryption and access control measures.

2.5 Summary

The project aims to develop and deploy a rainfall analysis and flood prediction website that is robust and user-friendly, with a focus on providing timely information and warnings to communities vulnerable to flooding. Key features include real-time data integration and alert systems to keep users informed, alongside a strong emphasis on accessibility.

SYSTEM ANALYSIS

3.1 Introduction

The increasing frequency and severity of flooding events necessitate a comprehensive approach that not only addresses the immediate aftermath but also focuses on preemptive measures. In response to the pressing need for effective flood management and mitigation strategies, our project endeavors to provide a comprehensive solution. Drawing inspiration from the imperative to empower communities with timely information. Our aim is to facilitate informed decision-making, proactive measures, and community-wide preparedness, ultimately mitigating the adverse impacts of floods and reducing associated losses.

3.2 Proposed System

We have developed a user-friendly interface accessible to the public. This interface offers real-time insights into river water levels, alongside a proactive notification mechanism designed to alert users of potential flood risks within the upcoming 12-month period. Moreover, our system goes beyond mere prediction, enabling users to delve into historical trends of rainfall in their region and the incorporation of rainfall pattern visualization at the Sub-Division level enhances the system's efficacy in offering localized insights.

3.3 Limitations

While the proposed system offers valuable functionalities for flood risk management and public notification, there are some limitations to acknowledge:

- **Limited Data Source for Flood Prediction:** The system currently relies on data sources that is provided on governmental sites, but it excludes satellite imagery. Satellite imagery can provide valuable insights into factors like land cover, terrain elevation, and real-time flood inundation extent. Integrating satellite data could potentially improve the accuracy and granularity of flood predictions.

- **Absence of Real-Time Chatbot Support:** During critical flood situations, immediate and personalized communication can be crucial. The current system lacks a live chatbot function. Integrating a chatbot could offer real-time support to users by answering their questions, providing evacuation guidance, and directing them to essential resources.
- **Limited Prediction Window:** While the system offers a 12-month prediction window, this may not be sufficient for all scenarios. Flash floods, caused by intense localized rainfall, can occur with very little warning. The system might not be able to provide adequate lead time for these events.
- **Data Accuracy and Coverage:** The accuracy of flood predictions heavily relies on the quality and comprehensiveness of the underlying data. It's important to acknowledge any limitations in data coverage, particularly for remote or sparsely monitored areas.
- **False Positives and Negatives:** Flood prediction models may generate false positives (predicting a flood that doesn't occur) or false negatives (missing a real flood event). Transparency about these limitations can help manage user expectations.

3.4 Summary

Our system analysis delved into a flood risk management and public notification system intended to equip citizens with the knowledge to prepare. The system provides a window into river health by offering real-time and historical flow data, alongside 12-month flood forecasts and visualizations of historical trends and rainfall patterns. This transparency empowers proactive measures in anticipation of potential floods. However, the system isn't without its shortcomings. Currently, it doesn't utilize satellite imagery, lacks a live chatbot for emergencies, and faces hurdles with data accuracy and potential prediction errors. Addressing these limitations, while also considering factors like the limited prediction window, model dependence, human error, accessibility, and language barriers, will be essential for building a more resilient and inclusive system that effectively mitigates flood risks. This analysis serves as the foundation for system design and development, ensuring it fulfills its objectives of public notification and flood risk management.

SYSTEM DESIGN AND IMPLEMENTATION

4.1 Introduction

Leveraging the insights gained from the system analysis, we now embark on the design and implementation phase of this flood risk management and public notification system. Our focus will be on translating the identified functionalities into a user-friendly and accessible platform. This stage will involve meticulously crafting the system architecture, selecting appropriate technologies, and meticulously integrating various components to ensure seamless operation.

4.2 Module Design and Implementation

4.2.1 main.py

System Design:

- **Flask Web Application:** The module is built using Flask, a Python web framework, to handle HTTP requests and responses. Flask routes are defined for different functionalities like displaying the homepage, flood and rainfall analysis results, and about page.
- **Integration with Other Modules:** Import driver, rainfall, and alerter modules, indicating integration with other components responsible for flood prediction, rainfall analysis, and alert generation.
- **User Interface:** Utilizes HTML templates (index.html, about.html, flood_entry.html, rain_entry.html, flood_result.html, rain_result.html) for rendering user interfaces for different pages. The user interface allows users to input data for flood prediction and rainfall analysis and displays the results accordingly.

Implementation:

- **Initialization:** Creates a Flask application instance and sets a secret key for session management. Defines routes for different pages like the homepage, about page, flood prediction page, rainfall analysis page, and result pages.
- **Homepage and Navigation:** Renders the homepage (index.html) when the root URL is accessed. Provides navigation links to other pages such as flood prediction, rainfall analysis, and about page.
- **Flood Prediction:** Provides functionality to refresh flood warning data .Renders the flood prediction homepage (flood_home.html) which displays flood alerts based on data from the alerter module.
- **Rainfall Analysis:** Renders the rainfall analysis homepage (rainfall_home.html) where users can input data for rainfall analysis. Handles user input for rainfall analysis (rainfall_result route) and displays the results (rain_result.html) using the rainfall module.
- **Error Handling:** Performs basic input validation and error handling for user inputs.
Displays flash messages for empty input fields.
- **Server Initialization:** Starts the Flask application server in debug mode.

4.2.2 rainfall.py

System Design:

- **Data Loading and Preprocessing:** Reads rainfall data from a CSV file (Sub_Division_IMD_2021.csv). Handles missing values by removing rows with NaN values and filling remaining missing values with column means.
- **Data Generation Functions:** Generates input-output pairs for a given year and region by sliding a window over the monthly rainfall data.

- **Neural Network Model:** Utilizes a Recurrent Neural Network (RNN) implemented with Keras for rainfall prediction. Defines the RNN model architecture with dense layers. Compiles the model with a mean squared error loss function and Adamax optimizer.
- **Training and Prediction:** Trains the RNN model using rainfall data for a specific region. Predicts rainfall for a given year and region using the trained model. Computes Mean Absolute Error (MAE) and Explained Variance Score for evaluation.
- **Graph Plotting and Saving:** Plots the ground truth vs. predicted rainfall for visualization. Saves the generated rainfall graph as an image (rainfall.png) in the static directory.

Implementation:

- **Data Handling:** Uses pandas for data manipulation and cleaning. Fills missing values with column means to ensure complete data for analysis.
- **Model Training and Evaluation:** Implements a CNN model using Keras for rainfall prediction. Trains the model using historical rainfall data. Evaluates the model's performance using MAE and Explained Variance Score.
- **Graph Plotting:** Visualizes the predicted vs. actual rainfall using bar plots for each month. Saves the generated graph as an image for later display in the web application.

4.2.3 driver.py

System Design:

- **Data Processing:** Imports necessary modules (discharge, flood_runoff, daily_runoff, weekly_runoff, model) for data processing, flood prediction, and model utilization. Reads input data from an Excel file (filename.xlsx).
- **Date Validation:** Defines a function fut_cal to determine if the given date is in the future compared to the last date in the dataset.

- **Flood Prediction:** Implements the drive function to perform flood prediction based on the given filename and user-provided date. Handles both existing data prediction and future data prediction scenarios.
- **Existing Data Prediction:** Processes existing data if the user-provided date is within the dataset's range. Utilizes existing data columns such as discharge, flood runoff, daily runoff, and weekly runoff for prediction. Returns prediction results including discharge, flood runoff, daily runoff, weekly runoff, mean absolute error, predicted flood status, and actual flood status.
- **Future Data Prediction:** Generates future data predictions if the user-provided date is beyond the dataset's range. Utilizes forecasted data for discharge, flood runoff, daily runoff, and weekly runoff for prediction. Returns prediction results including discharge, flood runoff, daily runoff, weekly runoff, predicted flood status, and actual flood status.

Implementation:

The implementation involves reading data from an Excel file, validating the user-provided date, and performing flood prediction based on existing or future data.

Prediction results are returned in a dictionary format.

4.2.4 model.py

System Design:

- **Data Preprocessing:** Imports necessary libraries like pandas for data manipulation and MinMaxScaler for feature scaling. Reads input data from an Excel file. Processes the data by filling missing values with column means, encoding target labels, and extracting additional features like day, month, and year.
- **Data Splitting and Resampling:** Splits the dataset into training and testing sets. Balances the dataset using Synthetic Minority Over-sampling Technique (SMOTE) to handle class imbalance.
- **Model Training:** Loads a pre-trained model from disk using joblib's load function. Utilizes the loaded model to make predictions on the test data. Evaluates model performance using Mean Absolute Error (MAE).

- **Prediction Function:** Defines a function (predicting) to make predictions on future data. Processes the input data, predicts the flood occurrence using the trained model, and returns the predictions.

Implementation:

The implementation involves data preprocessing, model loading, prediction, and evaluation. The model employs a pre-trained classifier, which is loaded from a disk and used for making flood predictions.

4.2.5 alerter.py

System Design:

- **Data Loading and Processing:** Imports necessary libraries like pandas for data manipulation. Reads forecasted river level data from CSV files for different rivers. Processes the data by filling missing values with column means.
- **Flood Classification:** Implements a function (flood_classifier) to classify the likelihood of flood occurrence based on discharge, flood runoff, daily runoff, and weekly runoff forecast data. Utilizes Linear Discriminant Analysis (LDA) for classification.
- **Alert Generation:** Defines a function (alerting) to generate flood alerts based on the classification results. Identifies rivers with predicted flood occurrence and adds them to the list of alerts.
- **Data Creation:** Implements a function (data_creator) to create combined forecast data for discharge, flood runoff, daily runoff, and weekly runoff for each river.
- **Data Saving:** Saves the generated flood alerts and forecasted river level data to CSV files for future reference.

Implementation:

The implementation involves data loading, processing, flood classification, alert generation, and data saving. Flood classification is performed using LDA, and alerts are generated based on the classification results.

4.2.6 discharge.py

System Design:

- **Data Loading and Processing:** Imports necessary libraries like pandas for data manipulation and statsmodels for time series analysis. Reads raw discharge data from an Excel file and preprocesses it by filling missing values with column means.
- **Resampling and Scaling:** Resamples the data daily and scales the discharge values using MinMaxScaler to ensure uniformity and comparability.
- **Dataset Preparation:** Divides the data into training and testing sets based on the provided parameter wtd (0 for training, 1 for testing). Creates datasets suitable for modeling and forecasting using ARIMA.
- **Modeling and Forecasting:** Defines an ARIMA model using statsmodels ARIMA. Fits the ARIMA model to the training data and forecasts future discharge values. Generates forecasts for a specific number of time steps (adjustable as needed).
- **Evaluation and Visualization:** Evaluates the model performance using Mean Absolute Error (MAE) if wtd is set to 0 (training mode). Plots the actual and forecasted discharge values and saves the plot for visualization.
- **Data Saving:** Saves the forecasted discharge data to a CSV file for future reference.

Implementation:

The implementation involves data loading, processing, dataset preparation, modeling, forecasting, evaluation, visualization, and data saving.

ARIMA is utilized for time series forecasting, and discharge forecasts are generated based on the trained model.

4.2.7 flood_runoff.py

System Design:

- **Data Loading and Processing:** Imports necessary libraries like pandas for data manipulation and statsmodels for time series analysis. Reads raw flood runoff data from an Excel file and preprocesses it by filling missing values with column means.
- **Resampling and Scaling:** Resamples the data daily and scales the flood runoff values using MinMaxScaler to ensure uniformity and comparability.
- **Dataset Preparation:** Divides the data into training and testing sets based on the provided parameter wtd (0 for training, 1 for testing). Creates datasets suitable for modeling and forecasting using ARIMA.
- **Modeling and Forecasting:** Defines an ARIMA model using statsmodels ARIMA. Fits the ARIMA model to the training data and forecasts future flood runoff values. Generates forecasts for a specific number of time steps (adjustable as needed).
- **Evaluation and Visualization:** Evaluates the model performance using Mean Absolute Error (MAE) if wtd is set to 0 (training mode). Plots the actual and forecasted flood runoff values and saves the plot for visualization.
- **Data Saving:** Saves the forecasted flood runoff data to a CSV file for future reference.

Implementation:

The implementation involves data loading, processing, dataset preparation, modeling, forecasting, evaluation, visualization, and data saving. ARIMA is utilized for time series forecasting, and flood runoff forecasts are generated based on the trained model.

4.2.8 daily_runoff.py

System Design:

- **Data Loading and Procorts** necessary libraries like pandas for data manipulation and statsmodels for time series analysis. Reads raw daily runoff data from an Excel file and preprocesses it by filling missing values with column means.

- **Resampling and Scaling:** Resamples the data on a daily basis and scales the daily runoff values using MinMaxScaler to ensure uniformity and comparability.
- **Dataset Preparation:** Divides the data into training and testing sets based on the provided parameter wtd (0 for training, 1 for testing). Creates datasets suitable for modeling and forecasting using ARIMA.
- **Modeling and Forecasting:** Defines an ARIMA model using statsmodels ARIMA. Fits the ARIMA model to the training data and forecasts future daily runoff values. Generates forecasts for a specific number of time steps (adjustable as needed).
- **Evaluation and Visualization:** Evaluates the model performance using Mean Absolute Error (MAE) if wtd is set to 0 (training mode). Plots the actual and forecasted daily runoff values and saves the plot for visualization.
- **Data Saving:** Saves the forecasted daily runoff data to a CSV file for future reference.

Implementation:

The implementation involves data loading, processing, dataset preparation, modeling, forecasting, evaluation, visualization, and data saving. ARIMA is utilized for time series forecasting, and daily runoff forecasts are generated based on the trained model.

4.2.9 weekly_runoff.py

System Design:

- **Data Loading and Processing:** Reads the raw weekly runoff data from an Excel file and preprocesses it by filling missing values with column means. Converts the date column to a datetime format for time series analysis.

- **Resampling and Scaling:** Resamples the data on a weekly basis and scales the weekly runoff values using MinMaxScaler to normalize the data.
- **Dataset Preparation:** Divides the data into training and testing sets based on the provided parameter wtd (0 for training, 1 for testing). Constructs datasets suitable for modeling and forecasting using ARIMA.
- **Modeling and Forecasting:** Utilizes the ARIMA model from the statsmodels library for time series forecasting. Defines the ARIMA model order and fits it to the training data. Generates forecasts for a specific number of time steps (adjustable as needed).
- **Evaluation and Visualization:** Evaluates the model performance using Mean Absolute Error (MAE) if wtd is set to 0 (training mode). Plots the actual and forecasted weekly runoff values and displays the plot for visualization.
- **Data Saving:** Saves the forecasted weekly runoff data to a CSV file for future reference.

Implementation:

The implementation involves data loading, processing, dataset preparation, modeling, forecasting, evaluation, visualization, and data saving. ARIMA, a widely used time series forecasting method, is employed for generating weekly runoff forecast.

4.3 Templates Design

4.3.1 index.html

- **Document Structure:** Begins with conditional comments for Internet Explorer compatibility. Contains standard HTML structure with <head> and <body> sections.

- **Metadata:** Includes essential meta tags such as charset, viewport, and description. Specifies the title of the webpage and sets a favicon icon.
- **Stylesheets:** Imports various CSS files for styling, including normalize.css, animate.css, and custom stylesheets.
- **JavaScript:** Loads the Modernizr JavaScript library for feature detection. Provides conditional loading of HTML5 Shiv and Respond.js for IE compatibility.
- **Preloader and Scroll-to-Top Button:** Displays a preloader animation while the page content is loading. Includes a scroll-to-top button for easy navigation to the top of the page.
- **Header Area:** Features a top area with a background image and navigation bar. Navigation bar includes links to the home and about sections of the website.
- **Welcome Slider Area:** Contains a slider with two slides showcasing project title and descriptions. Each slide includes buttons for navigating to specific sections like flood prediction and rainfall analysis.
- **Blog Section:** Presents a descriptive paragraph highlighting the importance of flood prediction and rainfall analysis.
- **Footer Area: Divided** into top and bottom sections with optional content. Includes social media links and other relevant information.
- **JavaScript Files:** Loads various JavaScript files for interactive features like carousel, navigation, and contact forms.

4.3.2 about.html

The about.html template provides information about the project's team members and related details, along with navigation options for users.

- **Document Structure:** Follows the same HTML structure as the previous template. Includes conditional comments for Internet Explorer compatibility.
- **Metadata and Title:** Contains essential meta tags like charset, viewport, description, and keywords. Specifies the title of the webpage as "About".
- **Stylesheets and Custom CSS:** Imports various CSS files for styling, including normalize.css, animate.css, and custom stylesheets. Defines custom CSS styles for specific elements such as team members.
- **JavaScript and Libraries:** Loads the Modernizer JavaScript library for feature detection. Provides conditional loading of HTML5 Shiv and Respond.js for IE compatibility.
- **Preloader and Scroll-to-Top Button:** Displays a preloader animation while the page content is loading. Includes a scroll-to-top button for easy navigation.
- **Header Area:** Features a top area with a background image and navigation bar. Navigation bar includes links to the home and about sections of the website.
- **Welcome Area:** Displays a welcome message and breadcrumb navigation for the about section.
- **About Details Area:** Includes tabbed content for displaying company information and team details. Provides sections for displaying company history and team members.
- **Footer Area:** Similar to the footer in the previous template, divided into top and bottom sections with optional content. Contains social media links and other relevant information.

- **JavaScript Files:** Loads various JavaScript files for interactive features like carousel, navigation, and contact forms. Enhances user experience and functionality of the webpage.

4.3.3 rain_entry.html

The rain_entry.html template provides a user interface for users to input data for rainfall analysis, along with navigation options and other standard webpage features.

- **Document Structure:** The document structure is standard HTML5 with conditional comments for Internet Explorer compatibility.
- **Metadata and Title:** Meta tags include character encoding, viewport settings, and descriptions relevant to transportation and agency themes. The title of the webpage is set as "Rainfall Prediction".
- **Favicon:** The webpage uses a custom favicon, represented by the favicon.png image file.
- **Stylesheets:** Various CSS files are linked for styling, including normalize.css, animate.css, stellarnav.min.css, owl.carousel.css, bootstrap.min.css, and font-awesome.min.css.
- **JavaScript and Libraries:** The webpage includes the Modernizr JavaScript library for feature detection.
- **Inline Styles:** Inline CSS defines a blinking animation effect for certain elements.
- **Preloader and Scroll-to-Top Button:** A preloader animation is displayed until the page content is fully loaded.

- **Header Area:** The header section features a navigation bar with links to the home and about sections of the website.
- **Welcome Area:** Displays a welcome message with breadcrumb navigation indicating the user's location on the website.
- **Contact Form:** A contact form is provided for users to input data for rainfall analysis. Users can select a subdivision and year for which they want to analyze rainfall data. Upon submitting the form, the data is sent to the "rainfall_result" endpoint for processing
- **Footer Area:** The footer section is divided into top and bottom areas, possibly for additional content. No specific content is provided in the template for the footer.
- **JavaScript Files:** Various JavaScript files are loaded for interactive features such as carousel, navigation, contact forms, and sticky elements.

4.3.4 rain_result.html

The rain_result.html template provides a user interface to display the results of rainfall analysis, along with navigation options and other standard webpage features.

- **Document Structure:** The document structure follows HTML5 conventions with conditional comments for Internet Explorer compatibility.
- **Metadata and Title:** Meta tags include character encoding, viewport settings, and descriptions relevant to transportation and agency themes. The title of the webpage is set as "Rainfall Prediction".
- **Favicon:** The webpage uses a custom favicon, represented by the favicon.png image file.

- **Stylesheets:** CSS files `normalize.css`, `animate.css`, `stellarnav.min.css`, `owl.carousel.css`, `bootstrap.min.css`, and `font-awesome.min.css` are linked for styling.
- **JavaScript and Libraries:** The webpage includes the Modernizr JavaScript library for feature detection.
- **Preloader and Scroll-to-Top Button:** A preloader animation is displayed until the page content is fully loaded. A scroll-to-top button facilitates easy navigation to the top of the page.
- **Header Area:** The header section features a navigation bar with links to the home and about sections of the website.
- **Welcome Area:** Displays a welcome message with breadcrumb navigation indicating the user's location on the website.
- **Result Display Area:** Presents rainfall analysis results, including Mean Absolute Error (MAE) and Accuracy Score in a tabular format.
- **Footer Area:** Similar to the header, it's divided into top and bottom areas, likely for additional content. No specific content is provided in the template for the footer.
- **JavaScript Files:** Various JavaScript files are loaded for interactive features such as carousel, navigation, contact forms, and sticky elements.

4.3.5 `flood_entry.html`

The `flood_entry.html` template provides a user interface for users to input data for flood prediction and displays results or notices related to flood forecasting.

- **Document Structure:** The document follows HTML5 conventions with conditional comments for Internet Explorer versions.
- **Metadata and Title:** Meta tags include character encoding, viewport settings, and descriptions relevant to transportation and agency themes. The title of the webpage is set as "Flood Prediction".
- **Favicon:** The webpage uses a custom favicon represented by the favicon.png image file.
- **Stylesheet:** CSS files normalize.css, animate.css, stellarnav.min.css, owl.carousel.css, bootstrap.min.css, and font-awesome.min.css are linked for styling.
- **JavaScript and Libraries:** The webpage includes the Modernizr JavaScript library for feature detection. Multiple JavaScript files are loaded for various interactive features like carousel, navigation, contact forms, etc.
- **Custom CSS:** Custom CSS styles are defined inline within <style> tags for an element with the class "blink" to create a blinking effect.
- **Preloader and Scroll-to-Top Button:** A preloader animation is displayed until the page content is fully loaded.
- **Header Area:** The header section features a navigation bar with links to the home and about sections of the website.
- **Welcome Area:** Displays a welcome message with breadcrumb navigation indicating the user's location on the website. Includes a link to navigate back to the home page.
- **Contact Form Area:** Provides a form for users to input river selection and date for flood prediction. Upon submission, the form sends data to the flood_result

endpoint using the POST method. The form includes dropdowns to select the river and input for the date.

- **Notice Board:** Displays a list of notices or results related to flood prediction. The results are dynamically populated from the result list.
- **Footer Area:** Similar to the header, it's divided into top and bottom areas, likely for additional content. No specific content is provided in the template for the footer.

4.3.6 flood_result.html

The flood_result.html template presents the results of flood prediction calculations in a structured and visually appealing manner, with the ability to navigate back to the flood prediction home page.

- **Document Structure:** The document structure follows HTML5 conventions with conditional comments for Internet Explorer versions.
- **Metadata and Title:** Meta tags include character encoding, viewport settings, and descriptions relevant to transportation and agency themes. The title of the webpage is set as "Flood Prediction".
- **Favicon:** The webpage uses a custom favicon represented by the favicon.png image file.
- **Stylesheets:** CSS files normalize.css, animate.css, stellarnav.min.css, owl.carousel.css, bootstrap.min.css, and font-awesome.min.css are linked for styling.
- **JavaScript and Libraries:** The webpage includes the Modernizr JavaScript library for feature detection.

- **Preloader and Scroll-to-Top Button:** A preloader animation is displayed until the page content is fully loaded.
- **Header Area:** The header section features a navigation bar with links to the home and about sections of the website. It also includes a breadcrumb navigation indicating the user's location on the website with a link to return to the flood prediction home page.
- **Result Display Area:** Displays the results of flood prediction calculations in a tabular format. Each row of the table represents a property related to flood prediction, with corresponding result values dynamically populated from the result list.
- **Footer Area:** Similar to the header, it's divided into top and bottom areas, likely for additional content.

4.4 Technologies Used

- **Python**

Description: Python is a high-level, interpreted programming language known for its simplicity and readability. It offers a vast ecosystem of libraries and frameworks for various tasks such as web development, data analysis, machine learning, and more.

Usage: Python is the primary language used for server-side scripting, implementing the web application's backend logic, data processing, machine learning models, and server deployment.

- **HTML (Hypertext Markup Language)**

Description: HTML is the standard markup language for creating web pages and web applications. It provides the structure and content of web pages through a series of elements and tags.

Usage: HTML templates are rendered using Flask and Jinja2 to generate dynamic web content. HTML defines the structure of the web pages, including elements such as forms, buttons, and text

- **CSS (Cascading Style Sheets):**

Description: CSS is a style sheet language used for describing the presentation of a document written in HTML. It defines how HTML elements are displayed on screen, including layout, colors, fonts, and more.

Usage: CSS stylesheets are employed to customize the appearance and layout of HTML elements within the web application's frontend. CSS defines styles such as colors, fonts, margins, padding, and responsiveness.

- **Visual Studio Code (VS Code):**

Description: Visual Studio Code is a lightweight, yet powerful source code editor developed by Microsoft. It supports various programming languages and extensions, making it highly customizable and suitable for various development tasks.

Specific Use: The Python code, HTML, CSS, and JavaScript files will be written, debugged, and managed using VS Code. It provides features like syntax highlighting, code completion, debugging, and version control integration to streamline the development workflow.

4.5 Libraries used

- **Flask:**

Description: Flask is a lightweight WSGI web application framework in Python. It is designed to make getting started quick and easy, with the ability to scale up to complex applications.

Usage: Used to create web routes, render HTML templates, handle form submissions, and run the web application server.

- **Jinja2:**

Description: Jinja2 is a modern and designer-friendly templating engine for Python programming language. It is widely used for generating dynamic content in web applications.

Usage: Integrated with Flask to render HTML templates with dynamic content.

- **NumPy:**

Description: NumPy is a fundamental package for scientific computing with Python. It provides support for large, multi-dimensional arrays and matrices, along with a collection of mathematical functions to operate on these arrays.

Usage: Utilized for array manipulation and mathematical operations, especially in data preprocessing and machine learning.

- **Pandas:**

Description: Pandas is a fast, powerful, and flexible open-source data analysis and manipulation library built on top of Python. It provides data structures and functions to efficiently manipulate structured data.

Usage: Used for data manipulation, reading and writing data from/to various file formats, data preprocessing, and analysis.

- **Matplotlib:**

Description: Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python. It provides a MATLAB-like interface for plotting.

Usage: Employed for data visualization, especially in creating plots, charts, and graphs to visualize data insights.

- **Keras:**

Description: Keras is a high-level neural networks API, written in Python and capable of running on top of TensorFlow, CNTK, or Theano. It allows for easy and fast experimentation with deep learning models.

Usage: Utilized for building and training neural network models, especially for regression and classification tasks.

- **scikit-learn (sklearn):**

Description: Scikit-learn is a simple and efficient machine learning library for Python, providing tools for data mining and data analysis. It features various algorithms and utilities for classification, regression, clustering, dimensionality reduction, and more.

Usage: Employed for tasks such as data preprocessing, model evaluation, and building machine learning models.

- **SMOTE (Synthetic Minority Over-sampling Technique):**

Description: SMOTE is an oversampling technique used to handle class imbalance in datasets. It synthesizes new minority instances to balance the class distribution.

Usage: Applied in the context of machine learning models to address class imbalance by generating synthetic data points for the minority class.

- **Imbalanced-learn (imblearn):**

Description: Imbalanced-learn is a Python library offering a number of re-sampling techniques commonly used in datasets showing strong between-class imbalance.

Usage: Specifically used for implementing the SMOTE technique to handle class imbalance.

- **Statsmodels:**

Description: Statsmodels is a Python module that provides classes and functions for the estimation of many different statistical models, as well as for conducting statistical tests and statistical data exploration.

Usage: Utilized for time series analysis and forecasting, specifically with the ARIMA model.

- **Joblib:**

Description: Joblib is a set of tools to provide lightweight pipelining in Python. It provides utilities for saving and loading Python objects to/from disk.

Usage: Used for saving and loading trained machine learning models.

4.6 Methodologies Used

- **Time Series Forecasting with ARIMA Model:** AutoRegressive Integrated Moving Average (ARIMA) models are widely used in time series forecasting, particularly for hydrological data prediction. ARIMA models capture the temporal dependencies and patterns in the data by incorporating three components. ARIMA models are capable of capturing seasonality, trends, and other temporal patterns in the data, making them suitable for forecasting various hydrological parameters such as discharge, flood runoff, daily runoff, and weekly runoff.
- **Data Preprocessing and Scaling:** Data preprocessing involves preparing the dataset for model training by handling missing values, ensuring data consistency, and transforming features as needed.

Handling missing values: Techniques such as imputation are applied to replace missing values with a suitable estimate, such as the mean or median of the feature.

Scaling: Min-max scaling is used to normalize numerical features within a specific range, typically between 0 and 1. This ensures that all features contribute equally to model training and prevents features with larger scales from dominating the learning process.

Handling Imbalanced Datasets with SMOTE: Imbalanced datasets occur when one class (e.g., flood events) is significantly more prevalent than the others (e.g., non-flood events) in the dataset. Synthetic Minority Over-sampling Technique (SMOTE) is a popular technique used to address class imbalance by generating synthetic samples for the minority class. By balancing the dataset, SMOTE improves the model's ability to learn from the minority class and makes predictions more robust against imbalanced class distributions.

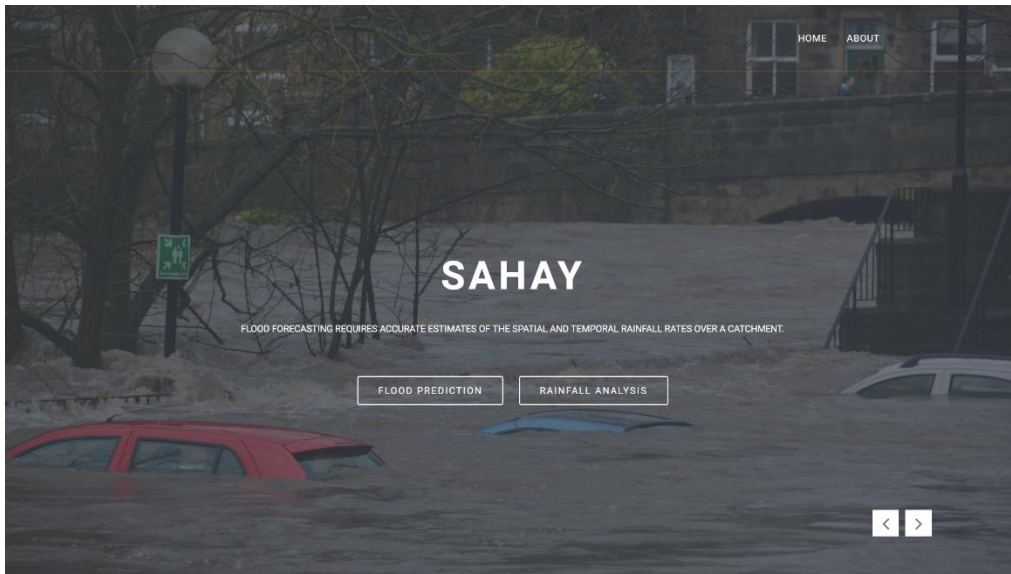
- **Model Evaluation with Mean Absolute Error (MAE):** Mean Absolute Error (MAE) is a commonly used metric for evaluating the performance of regression models,

including time series forecasting models like ARIMA. MAE measures the average absolute difference between the predicted values and the actual values in the dataset. A lower MAE indicates that the model's predictions are closer to the actual values, reflecting better accuracy and predictive performance. MAE provides a straightforward measure of model performance that is easy to interpret and compare across different models.

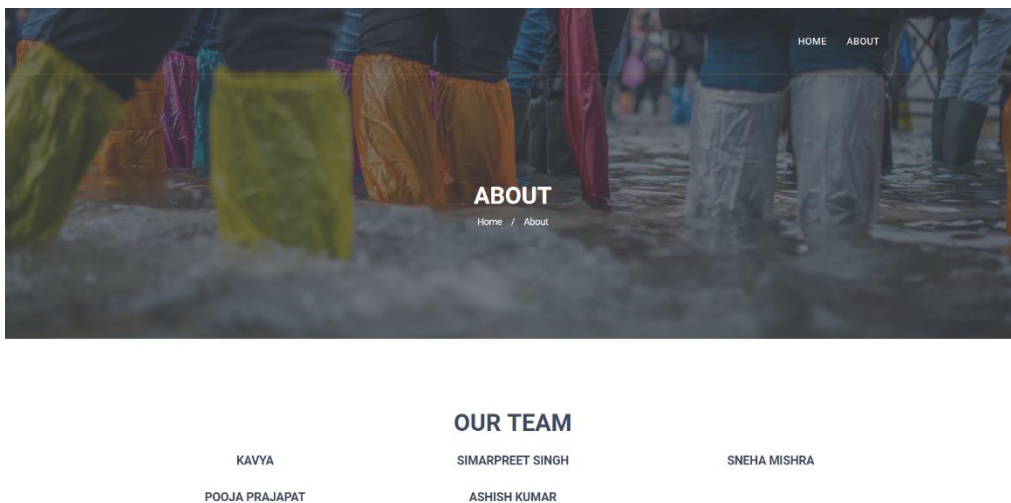
- **Classification Modeling with Linear Discriminant Analysis (LDA):** Linear Discriminant Analysis (LDA) is a classification technique used to classify observations into two or more classes based on predictor variables. In the flood prediction application, LDA is applied to classify hydrological data into binary classes, such as flood or no flood, based on specific thresholds. LDA identifies the linear combination of features that best separates the classes, enabling accurate classification and prediction of flood events.
- **Alerting and Decision Making:** Alerting mechanisms are implemented to notify stakeholders, such as emergency responders and local authorities, of potential flood events based on forecasted water levels and other relevant factors. Predictive models and real-time monitoring systems enable stakeholders to make informed decisions and take timely actions to mitigate the impact of floods. By providing early warnings and actionable insights, the flood prediction application supports proactive risk management and effective response strategies to minimize damage and save lives.
- **Data Visualization with Matplotlib:** Matplotlib is used for visualizing historical data, forecasted values, and model performance metrics through plots and graphs. Data visualization helps stakeholders understand patterns, trends, and fluctuations in hydrological data, facilitating better decision-making and risk assessment. Plots such as time series plots, scatter plots, and error plots provide valuable insights into the behavior of hydrological parameters and the performance of predictive models.

4.7 User Interface

index.html (fig.1)



about.html (fig. 2)



rainfall_entry.html (fig. 3)

HOME ABOUT

RAINFALL ANALYSIS

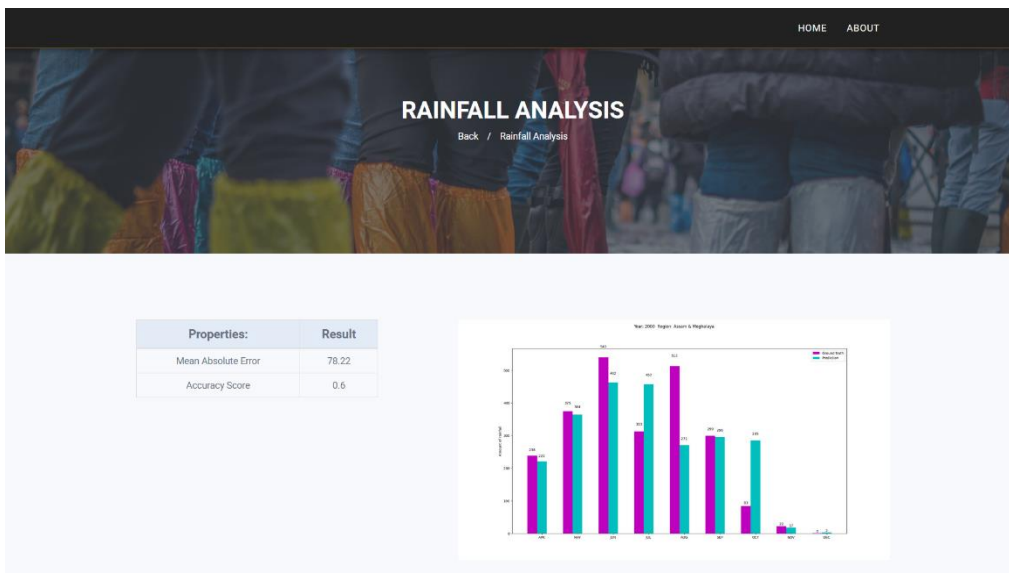
Home / Rainfall Analysis

Select Subdivisions: Assam & Meghalaya

Year: 2000

SEND

rainfall_result.html (fig. 4)



flood_entry.html (fig. 5)

HOMEABOUT

FLOOD PREDICTION

Home / Flood Prediction

Select River:

Cauvery

Date

03 / 12 / 2019

SEND

NOTICE BOARD

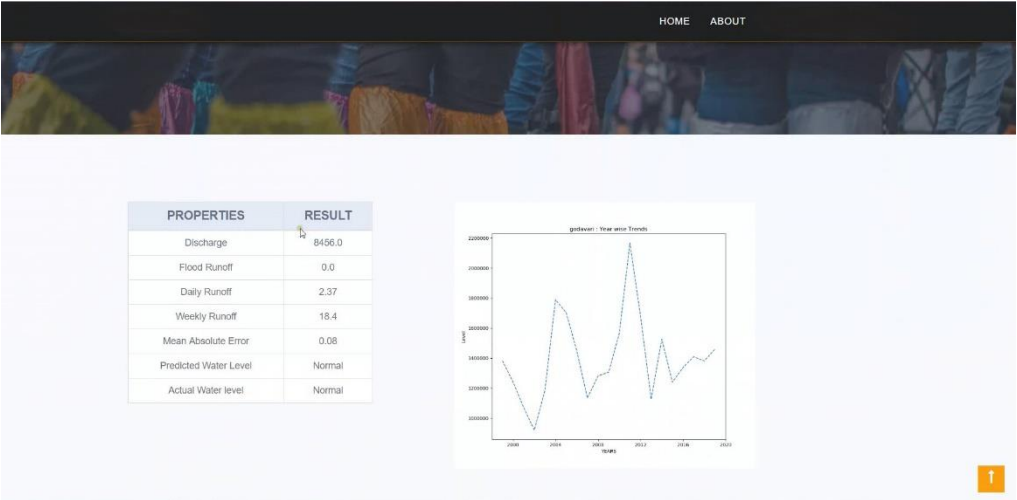
Flood ALERT for Cauvery

Flood ALERT for Mahanadi

Flood ALERT for Son

Reset

flood_result.html (fig. 6)



4.8 Summary

Our design and implementation phase translated the system analysis into a tangible rainfall analysis and flood risk management system. Python served as the foundation, while HTML, CSS, and JavaScript brought the user interface to life. For flood prediction, we leveraged various Python libraries to construct robust machine learning models. Modularity played a central role in the system's architecture. Divided into nine distinct modules, each with a well-defined purpose, the system promotes maintainability and scalability. A key focus was user-friendliness and accessibility, ensuring functionalities like real-time/historical river flow data, flood predictions via machine learning, and data visualization tools for historical trends and rainfall patterns. By prioritizing data accuracy, robust prediction models, and user accessibility, this system empowers communities to proactively prepare for and mitigate flood risks.

PERFORMANCE ANALYSIS

5.1 Introduction

This analysis evaluates rainfall analysis techniques and flood prediction models to improve understanding of hydrological processes and predict flood risks. It aims to contribute to the development of more effective strategies for water resource management and disaster preparedness, enhancing our ability to anticipate and manage flood events.

5.2 Performance Measures

Mean Absolute Error (MAE): Calculates the average absolute difference between observed and predicted rainfall values. It provides a measure of the magnitude of errors in the predictions, regardless of their direction.

The diagram illustrates the Mean Absolute Error (MAE) formula with the following components and annotations:

- MAE**: The symbol for Mean Absolute Error.
- =**: The equals sign.
- $\frac{1}{n}$** : A blue box containing the fraction 1 over n. A blue line points to it with the text "Divide by the total number of data points".
- $\sum$** : The summation symbol. Below it is the text "Sum of".
- $|$** : The absolute value bars.
- y** : The actual output value, enclosed in a green box. A green line points to it with the text "Actual output value".
- $-$** : The minus sign.
- $\hat{y}$** : The predicted output value, enclosed in an orange box. An orange line points to it with the text "Predicted output value".
- $|$** : The closing absolute value bar.
- The absolute value of the residual**: A bracket underneath the $y - \hat{y}$ term points to this text.

(fig. 7)

Accuracy: Measures the overall correctness of the model's predictions. It is calculated as the ratio of correctly predicted flood events to the total number of events.

$$\text{Accuracy} = \frac{\text{Correct predictions}}{\text{All predictions}}$$

It also provides the Mean Absolute Error (MAE) which is an evaluation metric used to calculate the performance regression models in machine learning. It also provides the accuracy of the data.

Our model uses the Statsmodel ARIMA (Autoregressive Integrated Moving Average) a statistical analysis model that uses time series data to better understand the dataset or predict the future trends to fit the data and make forecasts regarding the future values of flood runoff.

5.3 Performance Analysis

Performance analysis evaluates flood prediction and rainfall analysis models' accuracy and effectiveness, involving data collection, selection, validation, testing, and comparison. This helps researchers understand model strengths and limitations, improving flood risk management and disaster preparedness.

Our predicted model results show the adequate correctness of flood prediction and rainfall analysis. Our website analyzes various data sources such as river levels, sub divisional monthly rainfall and historical flood data which can predict the likelihood and severity of floods in specific areas with accuracy.

SAHAY provides user friendly interface by letting the user select the subdivision and year for which the user needs information about.

5.4 Summary

The website for rainfall analysis and flood prediction shows adequate correctness of flood prediction and rainfall analysis. However, improvements include data granularity and refining predictive algorithms. The website is crucial in mitigating risks from rainfall-induced flooding.

FUTURE ENHANCEMENT AND CONCLUSION

6.1 Introduction

SAHAY is committed to continuous enhancement and innovation in flood prediction and rainfall analysis, addressing the critical need for robust, adaptive systems in a rapidly changing environment and climate variability.

6.2 Limitation/Constraint of the system

The SAHAY website's rainfall analysis and flood prediction system has limitations and constraints. These include data availability and quality, modeling complexity, scale and scope, uncertainty and risk communication, resource constraints, dependence on external factors, and user engagement and adoption.

Data availability and quality are crucial for accurate predictions, while modeling complexity can be challenging due to computational resources and processing time. The system's effectiveness may vary depending on the scale and scope of its application, and communicating uncertainty effectively can be challenging. Additionally, external factors like government policies and community engagement can affect the system's performance and sustainability.

The project does not seem to meet its purpose when it comes to accurately predicting river floods since the input solely contains data of rivers-Godavari, Krishna, Son, Kaveri and Mahanadi while missing out on data of rivers like the Ganga that covers a large part of the country.

The system currently relies on data sources that is provided on governmental sites but it excludes satellite imagery. While the system offers a 12-month prediction window, this may not be sufficient for all scenarios. Flash floods, caused by intense localized rainfall, can occur with very little warning

6.3 Future Enhancements

For future improvement, we consider using other types of machine learning techniques, including the use of global optimization algorithms, for instance, the genetic algorithm that can expand the scope and scale of this study. In the future, this system will also be deployed on the web with additional features like sending warning SMS to authorities before the time that can help them plan safety measures for the people. We will include integrating satellite data that could potentially improve the accuracy and granularity of flood predictions.

Further development can also be made as we can include cloud deployment of the application.

Updating data utilizing live information from sensors. Also using advanced datasets taking into consideration climate change.

Integrating a chatbot could offer real-time support to users by answering their questions, providing evacuation guidance, and directing them to essential resources.

6.4 Conclusion:

Our model aids in data driven decision making by providing actionable insights from historical data and predictive analytics for flood prediction and rainfall analysis. By continuously innovating and staying at the forefront of technological advancements, we can ensure this system remains a valuable tool for flood risk mitigation. The ultimate goal is to empower communities around the world to prepare for potential flooding events with confidence, minimize damage, and safeguard the lives and livelihoods of their citizens.

The proposed Flood Prediction and Rainfall analysis has been successfully implemented and tested with the available data.

References

Comprehensive compilation of sources serves as the cornerstone for our work, fostering credibility and thoroughness. References are listed below.

- H. Shafizadeh-Moghadam, R. Valavi, H. Shahabi, K. Chapi, and A. Shirzadi, “Novel forecasting approaches using combination of machine learning and statistical models for flood susceptibility mapping,” *Journal of environmental management*, vol. 217, pp. 1-11, 2018.
- Tripathi G., Gupta R., “A New Approach for Rainfall Prediction using Artificial Neural Network”, *International Journal for Research in Applied Science & Engineering Technology (IJRASET)*, Volume 7 Issue VII, July 2019
- Majumdar N., Bhatnagar A., and Shukla S., “Using Linear Discriminant Analysis for Dimensionality Reduction for Predicting Anomalies of BGP data”, *International Journal of Innovative Technology and Exploring Engineering (IJITEE)*, 2019 Volume 8- 11, pp.1989-1995.
- Central Water Commission: <https://ffs.india-water.gov.in/>
- Department of Water Resources, River Development and Ganga Rejuvenation : <https://jalshakti-dowr.gov.in/flood-forecasting/>
- Abhijit Paul, Prodipto Das, Flood prediction model using artificial neural network, *International Journal of Computer Applications Technology and Research* Volume 3
- L. Xiaodong, G. Zhongyang, X. Ran, D. Xiaoyan, H. Yizhi, and Y. Shufeng, "Application of Artificial Neural Network in the Typhoon Flood Prediction System-A Case Study in Shanghai, China," in *Artificial Intelligence and Computational Intelligence (AICI)*, 2010 International Conference on, 2010, vol.1, pp. 191-195: IEEE.